CCNA 200-301 V2.0

30 Management Approaches, SNMP and Syslog

Part V — Wireless, Virtualisation and Operations

EXAM TOPICS

5.3 5.4 5.6

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Describe** the five network management approaches the blueprint names, and tell controller-based from automation-based.
- **Describe** how SNMP works, what its three versions differ on, and when to use a trap rather than an inform.
- **Interpret** any IOS syslog message: facility, severity, mnemonic and description.
- **Recite** the eight severity levels and explain what a logging threshold hides.

Before we start

Chapter 23 for secure management access and for why credentials in clear are unacceptable. Chapter 6 for UDP. Chapter 14 — you have already read syslog output there; this chapter makes it systematic.

KEY TERMS

Vocabulary for this chapter

- device-based
- controller-based
- cloud-based
- automation-based
- infrastructure as code
- SNMP manager
- agent
- MIB
- OID
- community string
- trap
- inform
- facility
- severity
- mnemonic

Each is defined where it first matters — not here.

ON THE EXAM

Three topics, three different verbs

This is what the blueprint actually asks for

5.3 *describe network management approaches (device-based, cloud-based, controller-based, automation-based, and infrastructure as code)* — five named approaches, so expect a matching or scenario question.

5.4 *describe the function of SNMP in network operations* — function, not configuration. What it does and how, not how to type it.

5.6 *interpret syslog message content, severity levels, and facilities* — **interpret**. You will be shown a log line and asked what it means. Section sec:syslog-read is built for exactly that and is the highest-value part of this chapter.

WATCH OUT

Controller-based against automation-based: the distinction that gets tested

They sound similar and are structurally different.

Controller-based inserts a *new system* between you and the network. The controller holds the intended state, and it is in the path permanently. Take it away and you lose central management.

Automation-based adds *tooling* outside the network. Ansible logs in over SSH and types commands faster and more consistently than you can. Take it away and every device still works exactly as before, because nothing about them changed.

One-line test: **does the network still have a management system when the tool is switched off?** Controller, yes — and you have lost it. Automation, no — and you have lost nothing but convenience.

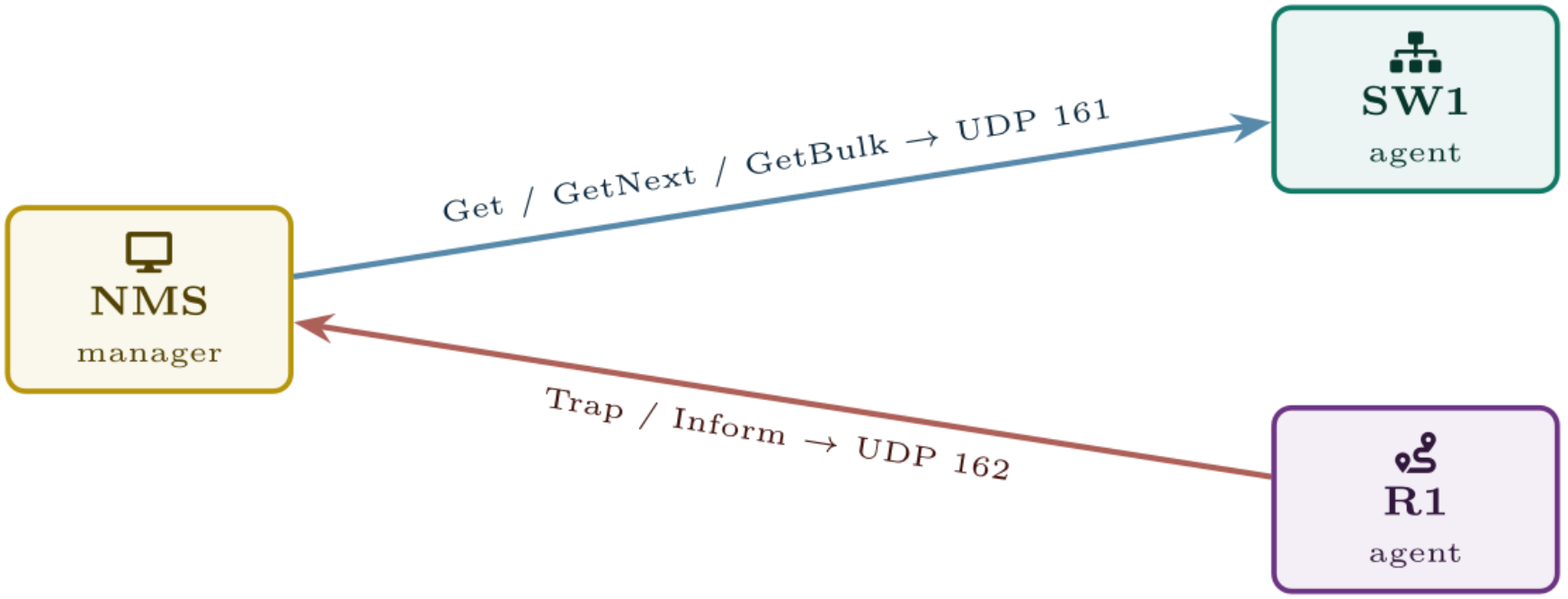
Infrastructure as code is a step beyond automation: not just *running* the change, but keeping the intended state in a repository that is reviewed, tested and versioned.

DEFINITION

SNMP

The Simple Network Management Protocol. A **manager** polls **agents** running on network devices for values held in a **MIB** — a tree of variables, each identified by a numeric **OID**. Agents listen on UDP 161; the manager receives unsolicited notifications on UDP 162.

SNMP



polling — the manager asks, on a schedule
notification — the agent speaks when something happens

SNMP works in both directions: the manager polls on port 161, and agents report events unprompted to port 162.

CONCEPT

Trap or inform

A trap that is lost is lost silently, and the event it described never reaches anyone. An inform costs a round trip and some agent memory, and survives a dropped packet.

Use informs for anything you would be sorry to miss — a device losing power redundancy, a security event. Use traps for high-volume, low-consequence notifications where losing one does not matter. This is a fair exam question and a real design decision.

WATCH OUT

“v2c is secure because the community string is not public”

It is not. The community string travels in clear text in every single polling packet, hundreds of times an hour, and anyone who can capture one packet has it. Renaming it from `public` to something obscure changes nothing.

And a read-write community string is equivalent to an enable password: an attacker holding one can reconfigure the device, and on many platforms can pull the entire running configuration. If SNMP must be v2c for a legacy tool, make it read-only, restrict it with an ACL to the management station’s address, and plan its removal.

CONFIGURE

SNMPv3, read-only, for a monitoring station

IOS · configuration mode

```
snmp-server view OPSVIEW iso included
snmp-server group OPSGROUP v3 priv read OPSVIEW
!
! auth protects integrity and identity; priv adds encryption.
! "priv" implies "auth" -- you cannot encrypt without authenticating.
snmp-server user nms OPSGROUP v3 auth sha AuthPassPhrase priv aes 128 PrivPassPhrase
!
snmp-server host 10.0.0.45 version 3 priv nms
snmp-server enable traps
!
! These two are free and save an hour when somebody has to find the
! device in a building.
snmp-server location Campus Block C, Rack 4, RU 22
snmp-server contact netops@example.edu
```

CONCEPT

What SNMP is actually used for

Three jobs, and the exam's phrase "in network operations" points at them.

- **Performance monitoring** — polling interface counters every few minutes to produce the traffic graphs everybody recognises.
- **Fault notification** — traps and informs for events that cannot wait for the next poll.
- **Inventory** — serial numbers, models, software versions and uptime collected across the estate.

What it is *not* good at is configuration. `Set` exists, but nobody manages configuration by SNMP; that is what Chapters 31 and the automation approaches are for.

Anatomy of an IOS message

```
Aug 27 09:14:22.331:%LINEPROTO-5-UPDOWN:

Line protocol on Interface GigabitEthernet0/1, changed state to down
```

One line off a real router. Everything a syslog message carries is in it — what is each piece?

Anatomy of an IOS message

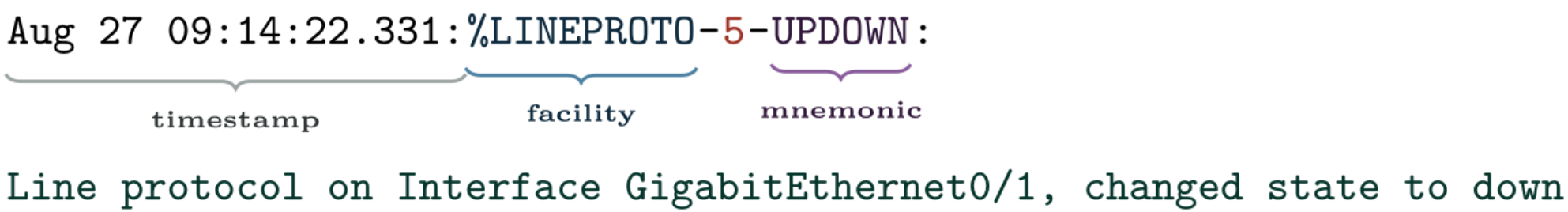
```
Aug 27 09:14:22.331:%LINEPROTO-5-UPDOWN:
```

timestamp

```
Line protocol on Interface GigabitEthernet0/1, changed state to down
```

The timestamp, and it is only trustworthy if NTP is.

Anatomy of an IOS message



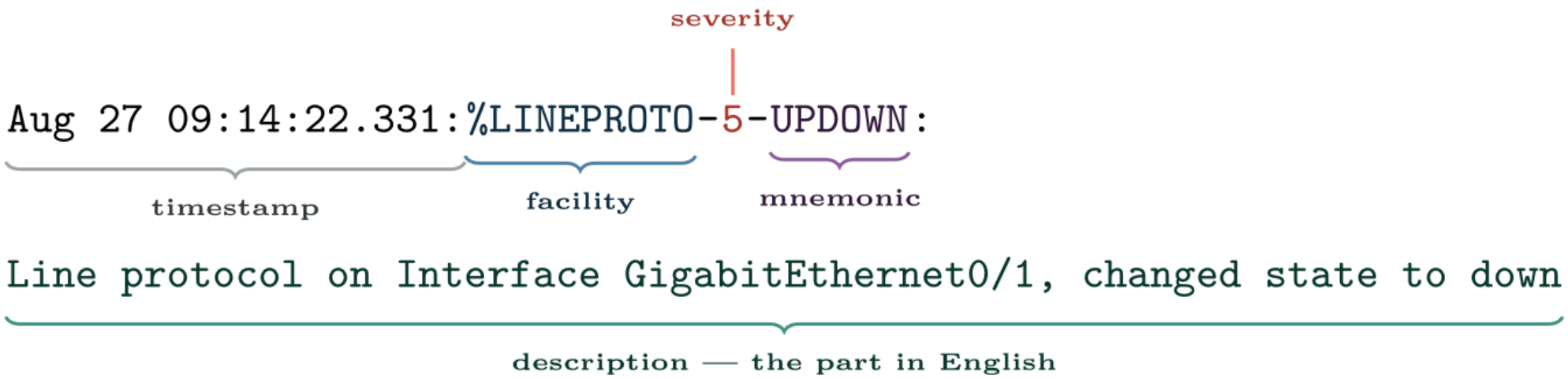
Facility says which subsystem spoke; mnemonic says what happened. Neither is the severity.

Anatomy of an IOS message


```
Line protocol on Interface GigabitEthernet0/1, changed state to down
```

The single digit between the two hyphens. That is the severity — not a count, not an interface number. This is the field the exam asks about.

Anatomy of an IOS message



And the rest is English, which is the only part most people read and the least useful part to filter on.

WATCH OUT

The severity is the digit in the middle, and it is not the number of anything else

In `%LINEPROTO-5-UPDOWN` the `5` is the severity level. In `%LINK-3-UPDOWN` the `3` is the severity level.

Those two messages describe the same interface going down and carry different severities because they concern different layers. Seeing both together is normal and tells you the physical link dropped, which took the line protocol with it. A `%LINEPROTO-5-UPDOWN` *without* a matching `%LINK-3-UPDOWN` means layer 1 stayed up and something above it failed — keepalives, an encapsulation mismatch, or a shutdown of a subinterface.

CONCEPT

Remembering the order

Every **A**wesome **C**isco **E**ngineer **W**ill **N**eed **I**ce-cream **D**aily — Emergency, Alert, Critical, Error, Warning, Notification, Informational, Debugging.

Two facts that follow and are examined: **0 is the most severe and 7 the least**, which is the opposite of most people's instinct. And configuring a logging destination at level *n* sends everything from 0 *through n* — so `logging console warnings` (level 4) also delivers levels 0 to 3, and silently discards 5, 6 and 7.

CONFIGURE

A sensible logging setup

IOS · configuration mode

```
! Without this every message is stamped only with uptime, which is
! useless for correlating across devices. Do this first, everywhere.
service timestamps log datetime msec localtime show-timezone
service timestamps debug datetime msec localtime show-timezone
!
! Keep a generous local buffer at informational, so the evidence is
! on the device even if the syslog server was unreachable.
logging buffered 65536 informational
!
! Console logging on a busy device makes the CLI unusable. Turn it
! down; the buffer and the server still have everything.
logging console warnings
logging monitor informational
!
! The central server. "trap" here means the syslog severity
! threshold, NOT an SNMP trap -- an unfortunate collision of terms.
logging host 10.0.0.45
logging trap notifications
logging source-interface Loopback0
```

CONFIGURE

A sensible logging setup (continued)

IOS · configuration mode

```
!
line con 0
 logging synchronous
line vty 0 15
 logging synchronous
```

WATCH OUT

logging trap has nothing to do with SNMP traps

It sets the *severity threshold for messages sent to a syslog server*. `logging trap notifications` means "send levels 0 to 5 to the host". The word is a historical accident and it confuses people every year.

`logging synchronous` is unrelated to both: it stops a log message printed mid-command from scrambling what you were typing. It changes nothing about what is logged and it will save your temper.

VERIFY

show logging | include Console|Buffer|Trap

SW1#

```
    Console logging: level warnings, 2201 messages logged
    Monitor logging: level informational, 0 messages logged
    Buffer logging:  level informational, 88214 messages logged
    Trap logging: level notifications, 41022 message lines logged
```

Common pitfalls

- **Reading severity backwards.** 0 is worst, 7 is least important.
- **Setting a threshold too high and concluding nothing is wrong.** The message may exist and be filtered. Check the levels before trusting an empty log.
- **No `service timestamps`.** Messages stamped with uptime cannot be correlated across devices or with a user's report.
- **Confusing `logging trap` with SNMP traps.**
- **Leaving console logging at debugging on a production device.** The CLI becomes unusable exactly when you need it.

Common pitfalls (continued)

- **SNMP v2c with a read-write community string.** That is an enable password sent in clear thousands of times a day.
- **No `logging source-interface`.** Messages arrive from whichever interface routing chose, and the server cannot reliably attribute them.
- **Relying on traps for events that matter.** Use informs.
- **Calling Ansible "controller-based".** It adds no control plane.

Ports keep going down overnight. The log “shows nothing”.

An access switch drops several user ports most nights. By morning they are back. The site’s monitoring shows the interfaces bouncing, but the network team reports that the device’s own log contains no explanation.

Ports keep going down overnight. The log “shows nothing”.

What would you check first — and what do you expect to see?

show logging | include level

```
SW1#
    Console logging: level errors, 14 messages logged
    Monitor logging: level errors, 0 messages logged
    Buffer logging:  level errors, 14 messages logged
    Trap logging: level errors, 14 message lines logged
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. Every destination is set to `errors`, which is severity 3. That delivers levels 0 to 3 and **discards 4, 5, 6 and 7**.

Look at what lives in the discarded range:

Recording enough to diagnose

IOS · configuration mode

```
service timestamps log datetime msec localtime show-timezone
logging buffered 65536 informational
logging trap notifications
logging console warnings
```

Making a device tell you what it is doing

Platform note. Packet Tracer has a syslog server and will carry tasks 1–6. SNMPv3 needs a real image and a real NMS, so tasks 7–9 want CML or GNS3 with an SNMP tool on a Linux host.

1. On a switch with default logging, run `show logging` and record every threshold before changing anything.
2. Enable `service timestamps` with date, time and milliseconds. Shut an interface and compare the message with one logged before the change.
3. Shut and unshut an interface and capture both messages. Identify the facility, severity and mnemonic in each, and explain why the two severities differ.
4. Set `logging buffered errors`. Repeat the shut and unshut. Record which of the two messages survives and explain why — this is the Break & Fix in miniature.

Making a device tell you what it is doing (continued)

1. Restore `logging buffered informational`, save a configuration change, and find the `%SYS-5-CONFIG_I` message. Note that it records *that* a change was made, not what it was.
2. Point `logging host` at a syslog server, set `logging trap notifications`, and confirm messages arrive. Then set it to `emergencies` and confirm they stop.
3. **SNMP.** Configure SNMPv3 with authPriv and poll the device's interface table from a Linux host. Record the OID you used.
4. Capture the polling traffic. Confirm you cannot read it. Then reconfigure for v2c, capture again, and **find the community string in the packet**. Write down what you saw.

Making a device tell you what it is doing (continued)

1. Configure a trap receiver, generate an event, and confirm receipt. Change the trap to an inform and observe the acknowledgement.
2. **Extension.** Write the logging and SNMP standard your site should apply to every device, with a one-line justification per command, and the one thing you would check first when someone says the log is empty.

CHECKPOINT 1 of 6

Name the five management approaches and give the one-line test that separates controller-based from automation-based.

Discuss · then advance

ANSWER

Checkpoint 1 of 6

Name the five management approaches and give the one-line test that separates controller-based from automation-based.

> Device-based, controller-based, cloud-based, automation-based, and infrastructure as code. The test: **does the network still have a management system when the tool is switched off?** For controller-based, yes — and you have just lost it. For automation-based, no — nothing about the devices changed.

CHECKPOINT 2 of 6

In %PM-4-ERR_DISABLE, identify the facility, the severity and the mnemonic, and say what the severity means.

Discuss · then advance

ANSWER

Checkpoint 2 of 6

In `%PM-4-ERR_DISABLE`, identify the facility, the severity and the mnemonic, and say what the severity means.

> Facility `%PM`, severity **4**, mnemonic `ERR_DISABLE`. Severity 4 is *warning*.

CHECKPOINT 3 of 6

Which severity levels does `logging buffered warnings` record, and which does it discard?

Discuss · then advance

ANSWER

Checkpoint 3 of 6

Which severity levels does `logging buffered warnings` record, and which does it discard?

> `warnings` is severity 4, so it records levels 0 to 4 and **discards** 5, 6 and 7 — notification, informational and debugging.

CHECKPOINT 4 of 6

State the difference between an SNMP trap and an inform, and when you would choose each.

Discuss · then advance

ANSWER

Checkpoint 4 of 6

State the difference between an SNMP trap and an inform, and when you would choose each.

> A trap is sent once over UDP and never acknowledged, so it can be lost silently. An inform is acknowledged and retransmitted until it is. Use informs for anything you would be sorry to miss, traps for high-volume low-consequence events.

CHECKPOINT 5 of 6

Why is SNMP v2c unsuitable regardless of how obscure the community string is?

Discuss · then advance

ANSWER

Checkpoint 5 of 6

Why is SNMP v2c unsuitable regardless of how obscure the community string is?

> Because the community string travels in **clear text** in every polling packet, hundreds of times an hour. Anyone who can capture one packet has it, so obscurity is irrelevant. A read-write string is equivalent to an enable password.

CHECKPOINT 6 of 6

What does `logging trap` configure, and what does it not?

Discuss · then advance

ANSWER

Checkpoint 6 of 6

What does `logging trap` configure, and what does it not?

> It sets the *severity threshold for messages sent to a syslog server*. It has nothing whatever to do with SNMP traps, despite the name.

CHAPTER SUMMARY

What to take away

- Five approaches: device-based (per box), controller-based (a controller holds intent), cloud-based (vendor-hosted management plane), automation-based (tooling drives existing interfaces), infrastructure as code (state in a reviewed repository).
- Controller-based adds a system to the network; automation-based does not. That is the distinction to be able to state.
- SNMP: manager polls agents on UDP 161 with Get, GetNext, GetBulk and Set; agents send traps and informs to UDP 162. Informs are acknowledged, traps are not.
- v1 and v2c send community strings in clear. Only v3 authenticates and encrypts. Use authPriv.

What to take away (continued)

- An IOS syslog message is `%FACILITY-SEVERITY-MNEMONIC: description`. The severity is the digit between the hyphens.
- Levels 0 to 7, emergency to debugging. **0 is worst.** A threshold of *n* records 0 through *n* and discards the rest.
- `service timestamps` first, always. `logging trap` is a syslog severity, not SNMP.
- An empty log describes the logging configuration, not the network.

Where we got to

- Five approaches: device-based (per box), controller-based (a controller holds intent), cloud-based (vendor-hosted management plane), automation-based (tooling drives existing interfaces), infrastructure as code (state in a reviewed repository).
- Controller-based adds a system to the network; automation-based does not. That is the distinction to be able to state.
- SNMP: manager polls agents on UDP 161 with Get, GetNext, GetBulk and Set; agents send traps and informs to UDP 162. Informs are acknowledged, traps are not.
- v1 and v2c send community strings in clear. Only v3 authenticates and encrypts. Use authPriv.

NEXT

Chapter 31 — Ansible and Configuration Management